

Algorithms and data structures I.

Linear Sorting

RadixSort

DistributionSort

CountingSort

OHW

minLevel(t:BinTree) : Z	
t ≠ 0	
return minLevRun(t,0)	return -1

minLevRun(t:Node*, sz: N) :Z			
t->left=0 ∧ t->right=0			
return sz	t->left≠0 ∧ t->right=0	t->left=0 ∧ t->right≠0	t->left≠0 ∧ t->right≠0
	return minLevRun(t->left, sz+1)	return minLevRun(t->right, sz+1)	return min(minLevRun(t->left, sz+1), minLevRun(t->right, sz+1))

minLevSziint(t:BinTree):Z				
t = 0				
return -1	t->left = 0 ∧ t->right = 0			
	return 0	sz1:=minLevSziint(t->left)		
		sz2:=minLevSziint(t->right)		
		sz1 = -1		
		return sz2+1	sz2 = -1	
			return sz1+1	return min(sz1,sz2)+1

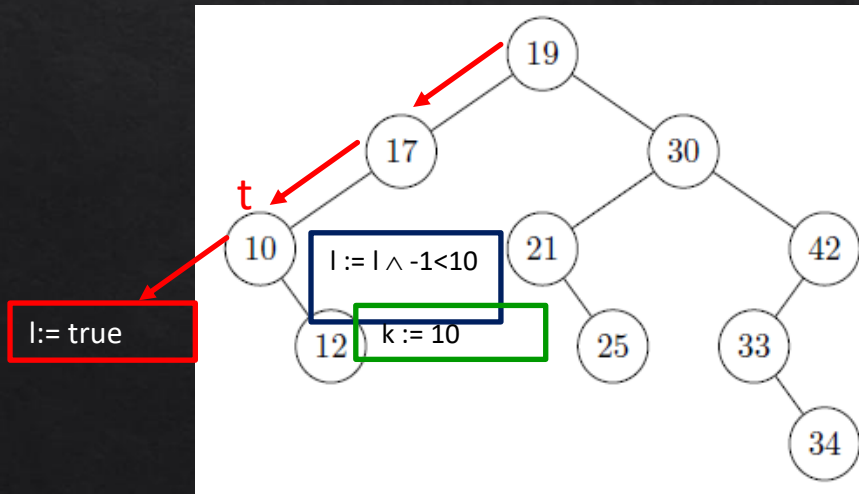
- ❖ Decide if binary search tree or not

checkBinSearchTree(t: Node*):B

$k := -1$

return checkInorder(t,k)

t k=-1



checkInorder(t:Node*, &k:Z) : B

t = 0

$l := \text{checkInorder}(t \rightarrow \text{left}, k)$

$l := l \wedge k < t \rightarrow \text{key}$

return true

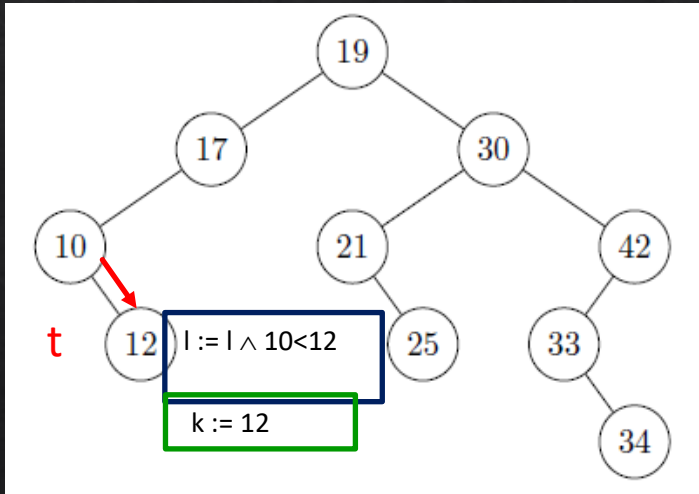
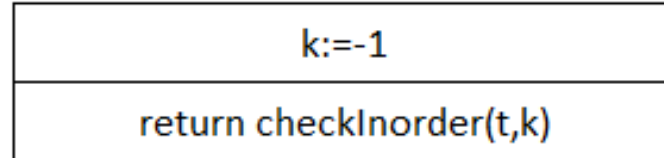
$k := t \rightarrow \text{key}$

return checkInorder(t->right,k)

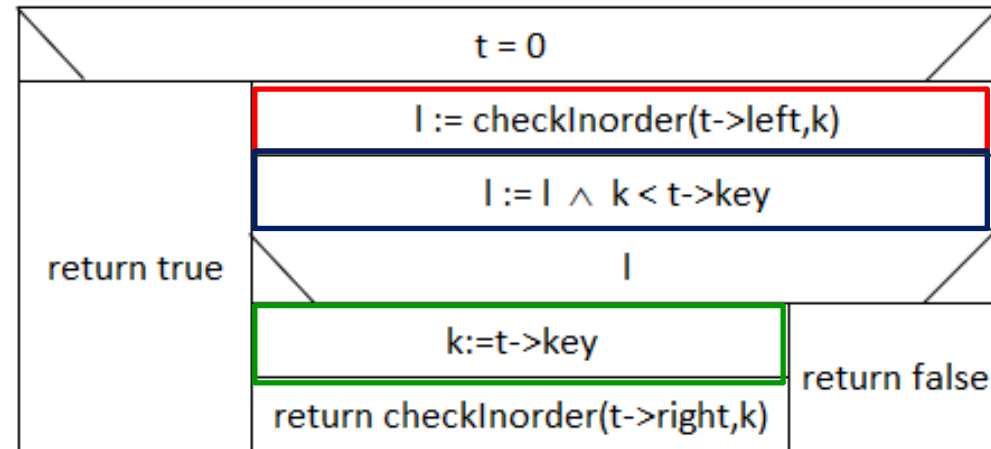
return false

- ❖ Decide if binary search tree or not

checkBinSearchTree(t: Node*):B

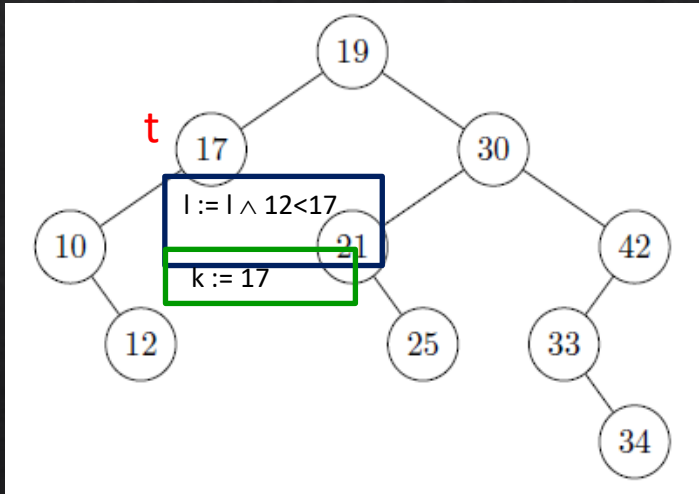
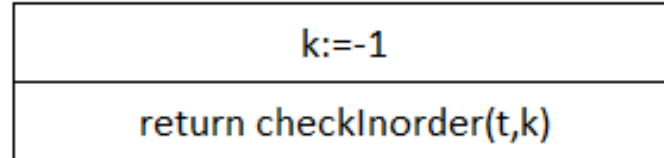


checkInorder(t:Node*, &k:Z) : B

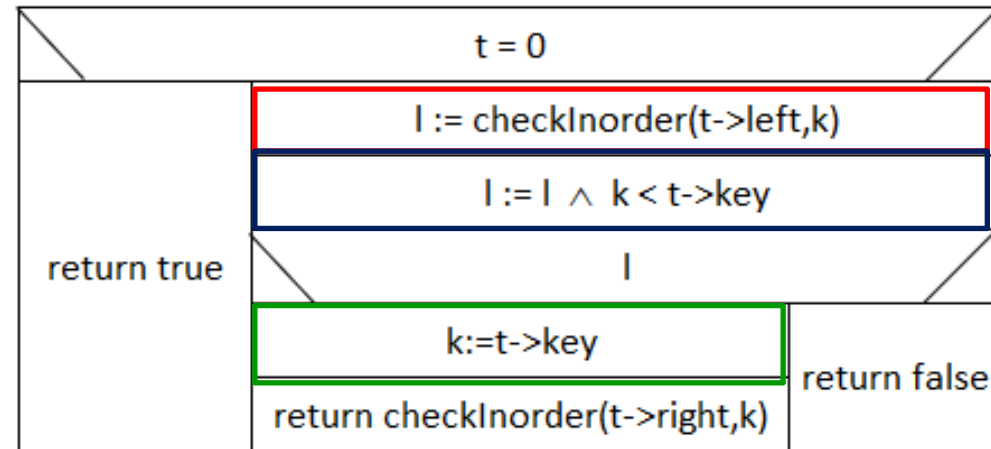


- ❖ Decide if binary search tree or not

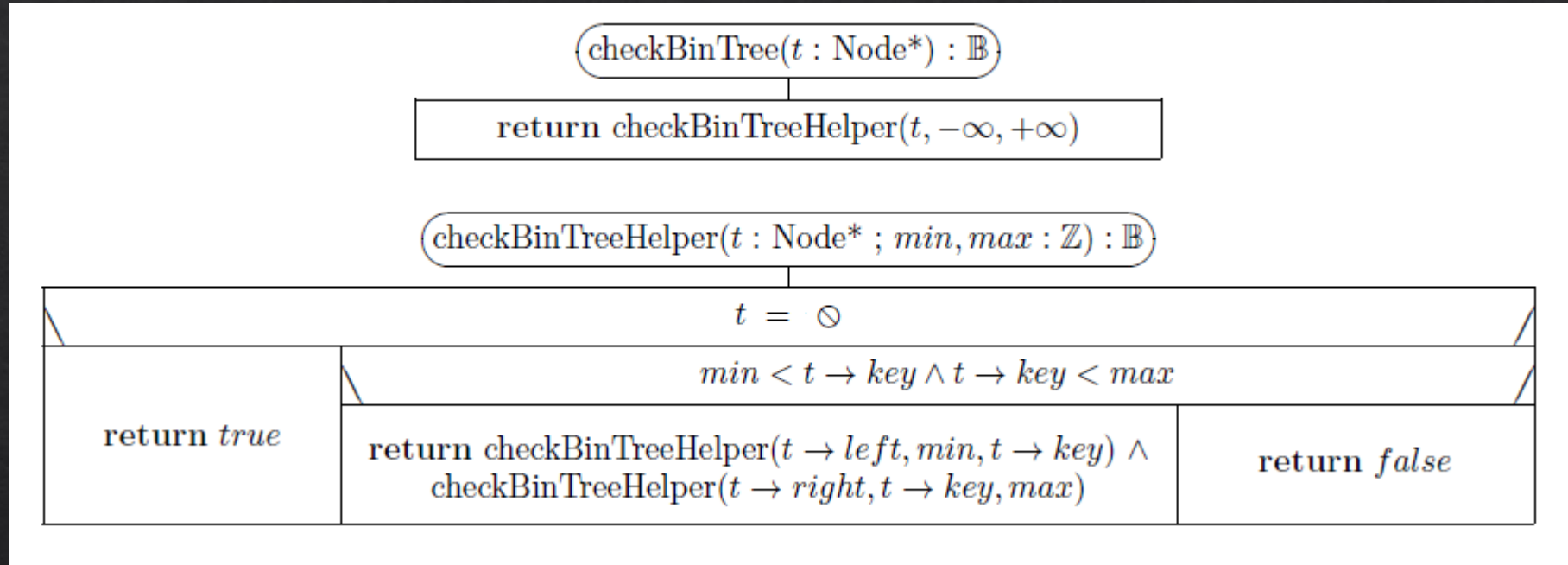
checkBinSearchTree(t: Node*):B



checkInorder(t:Node*, &k:Z) : B

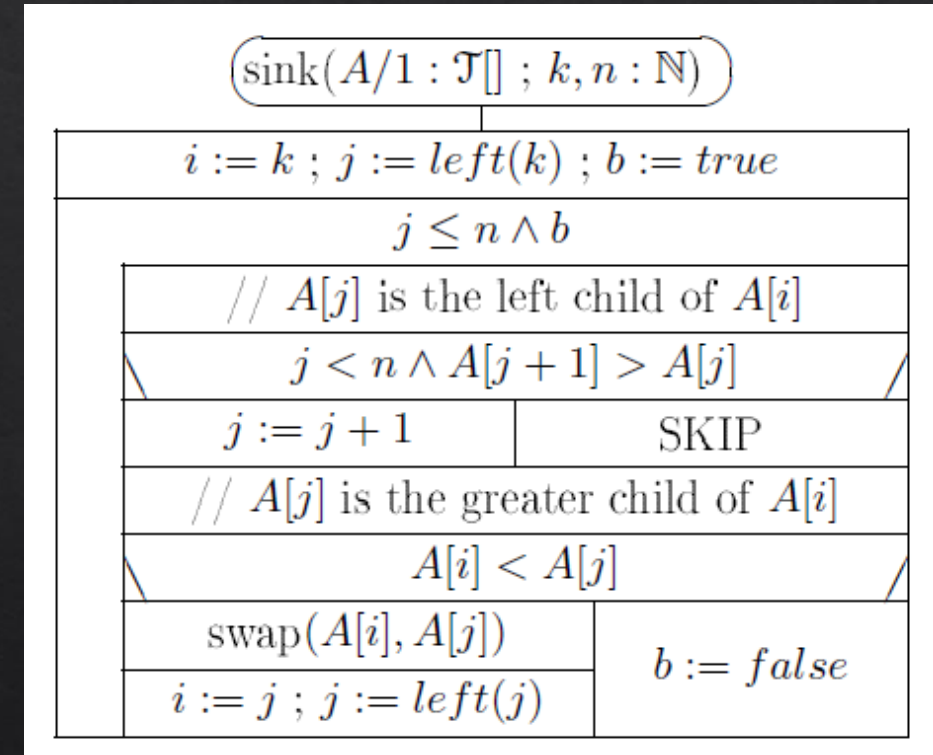
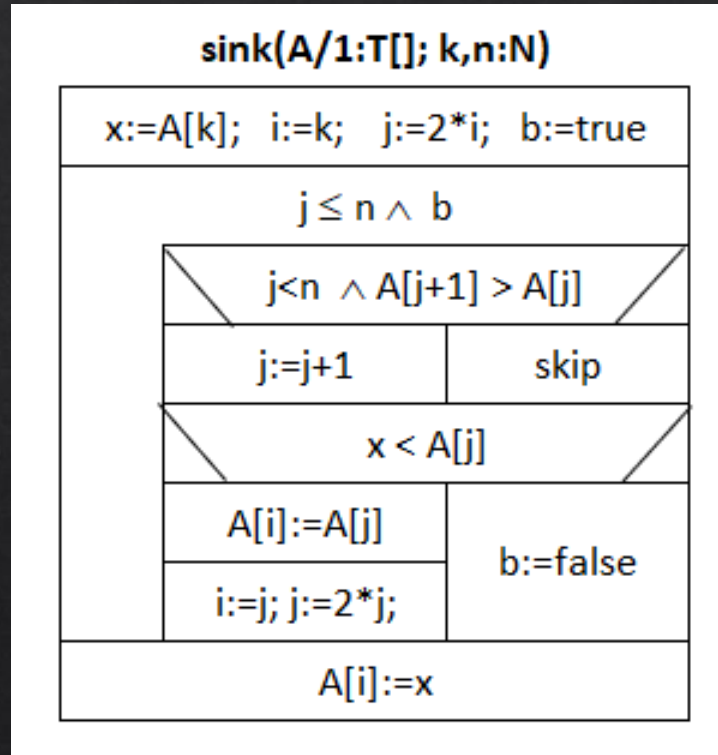


◇ Decide if binary search tree or not



OHW

Sinking without unnecessary swaps

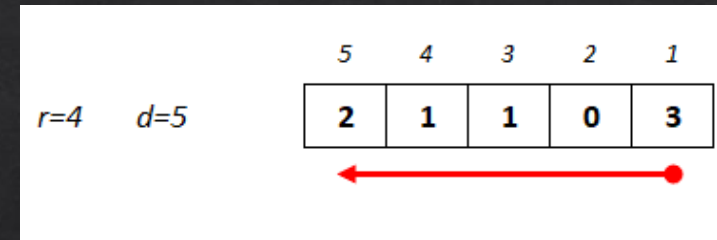
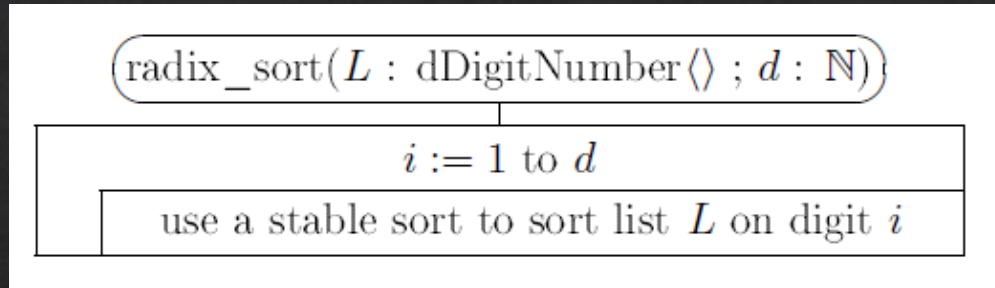


Lower bound of comparison based sorting

- ◇ 10.1. Theorem. Arbitrary sorting algo: $mT(n) = \Omega(n)$
- ◇ 10.3. Theorem. Any comparison based algo needs $MC(n) = \Omega(n \lg n)$ comparisons.
- ◇ 10.4. Theorem. Any comparison based sorting $MT(n) = \Omega(n \lg n)$.
- ◇ Note that Merge sort and heap sort have $O(n \lg n)$ as an upper bound, and $MT(n) = \Omega(n \lg n)$ lower bound. This means that $MT(n) = \Theta(n \lg n)$.

Radix sort principle

- ◆ Utilize the fact that our numbers are unsigned r based numerical systems, and have d digits, at most.
- ◆ e.g.: $r=4$, $d=5$: 21103 a possible key.
- ◆ In general:



- ◆ L is an abstract list, `dDigitNumber` is a number defined as above.
- ◆ How is this linear?
- ◆ Why require stability?

Distribution sort

- ◆ Sort the list L
- ◆ We distribute the elements of L into r helper lists in a stable manner (meaning we append at the end).
- ◆ Concatenate the lists one after the other from the back to the front.

$\text{distributing_sort}(L : \mathcal{T}\langle \rangle ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$B : \mathcal{T}\langle \rangle[r] //$ array of lists, i.e. bins

$k := 0$ to $r-1$

Let $B[k]$ be empty list $//$ init the array of bins

L is not empty

Remove the first element x of list L

Insert x at the end of $B[\varphi(x)] //$ stable distribution

$k := r-1$ downto 0

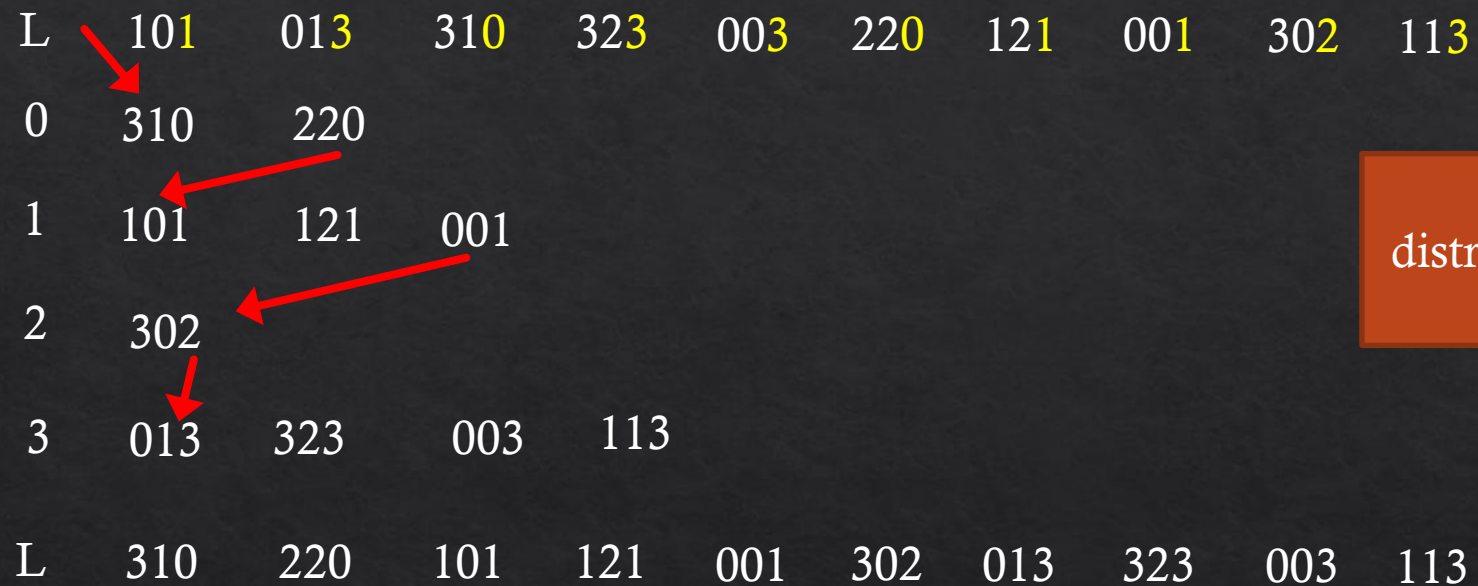
$L := B[k] + L //$ append $B[k]$ before L

Distribution sort as a radix sort

- ◆ The input has to be a linked list!
- ◆ We iterate over the elements based on their radix, starting from the least significant digit.
- ◆ In case of numeral system we need $0..r-1$ list, distribute elements into their corresponding list by appending at the very end (we might need a pointer to pointing at the end of the list)
- ◆ Then append the lists one after the other.
- ◆ Example for the list below: ($r=4$, $d=3$):

$L = \langle 101, 013, 310, 323, 003, 220, 121, 001, 302, 113 \rangle$

$L = \langle 101, 013, 310, 323, 003, 220, 121, 001, 302, 113 \rangle$



1. iteration:
distribute elements based on their
3rd (last) digit.

$L = \langle 310, 220, 101, 121, 001, 302, 013, 323, 003, 113 \rangle$

$L = \langle 310, 220, 101, 121, 001, 302, 013, 323, 003, 113 \rangle$

L	310	220	101	121	001	302	013	323	003	113
0	101	001	302	003						
1	310	013	113							
2	220	121	323							
3										

2. iteration:
Sorting based on the second digit
from the right.

L 101 001 302 003 310 013 113 220 121 323

$L = \langle 101, 001, 302, 003, 310, 013, 113, 220, 121, 323 \rangle$

$L = \langle 101, 001, 302, 003, 310, 013, 113, 220, 121, 323 \rangle$

L	101	001	302	003	310	013	113	220	121	323
0	001	003	013							
1	101	113	121							
2	220									
3	302	310	323							

3. iteration:
Last digit from the right

L	001	003	013	101	113	121	220	302	310	323
---	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

$L = \langle 001, 003, 013, 101, 113, 121, 220, 302, 310, 323 \rangle$

Implementation for C2Ls

$\text{radix_sort}(L : E2^* ; d, r : \mathbb{N})$

$BinHead : E2[r]$ // the headers of the lists representing the bins

$B : E2^*[r]$ // pointers to the headers

$i := 0$ to $r - 1$

$B[i] := \&BinHead[i]$ // Initialize the i th pointer.

$i := 1$ to d

$\text{distribute}(L, i, B)$ // Distribute L on the i th digits of keys.

$\text{gather}(B, L)$ // Gather form the bins back into L

$\text{gather}(B : E2^*[r] ; L : E2^*)$

$i := 0$ to $r - 1$

$\text{append}(L, B[i])$ // add to the end of L the elements form $B[i]$

$\text{distribute}(L : E2^* ; i : \mathbb{N} ; B : E2^*[r])$

$L \rightarrow next \neq L$

$p := L \rightarrow next ; \text{out}(p)$

$\text{precede}(p, B[\text{digit}(i, r, p \rightarrow key)])$

$\text{append}(L, Bi : E2^*)$

$Bi \rightarrow next \neq Bi$

$p := L \rightarrow prev ; q := Bi \rightarrow next ; r := Bi \rightarrow prev$

$p \rightarrow next := q ; q \rightarrow prev := p$

$r \rightarrow next := L ; L \rightarrow prev := r$

$Bi \rightarrow next := Bi ; Bi \rightarrow prev := Bi$

SKIP

Implementation for S1Ls

radix_sort(&L:E1*,d,r:N)

First:E1*[r]; Last:E1*[r]

i = 1 to d

i = 0 to r-1

First[i]:=Last[i]:=0

distribute(L,i,First,Last)

gather(L,First,Last)

distribute(&L:E1*, i:N, First:E1*[r]; Last:E1*[r])

L ≠ 0

p:=L; L:=L->next

k:=digit(i,r,p->key)

First[k] = 0

First[k]:=p

Last[k]->next:=p

Last[k]:=p; p->next:=0

gather(&L:E1*,First:E1*[r],Last:E1*[r])

L := 0

i = r-1 downto 0

First[i] ≠ 0

Last[i]->next := L

L := First[i]

skip

Counting sort

- ◇ Input: A array
- ◇ Output: B array
- ◇ Contains 0..r-1 integers (or can be mapped into them)

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$C : \mathbb{N}[r]$ // counter array
$k := 0$ to $r-1$
$C[k] := 0$ // init the counter array
$i := 0$ to $n-1$
$C[\varphi(A[i])]++$ // count the items with the given key
$k := 1$ to $r-1$
$C[k] += C[k-1]$ // $C[k] :=$ the number of items with key $\leq k$
$i := n-1$ downto 0
$k := \varphi(A[i])$ // $k :=$ the key of $A[i]$
$C[k]--$ // The next one with key k must be put before $A[i]$ where
$B[C[k]] := A[i]$ // Let $A[i]$ be the last of the {unprocessed items with key k }

- ◇ First step:
- ◇ Count the occurrences $C[0..r-1]$ of the numbers
- ◇ Cumulate the result.

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$C : \mathbb{N}[r]$ // counter array

$k := 0$ to $r-1$

$C[k] := 0$ // init the counter array

$i := 0$ to $n-1$

$C[\varphi(A[i])]++$ // count the items with the given key

$k := 1$ to $r-1$

$C[k] += C[k-1]$ // $C[k]$:= the number of items with key $\leq k$

$i := n-1$ downto 0

$k := \varphi(A[i])$ // k := the key of $A[i]$

$C[k]--$ // The next one with key k must be put before $A[i]$ where

$B[C[k]] := A[i]$ // Let $A[i]$ be the last of the $\{\text{unprocessed items with key } k\}$

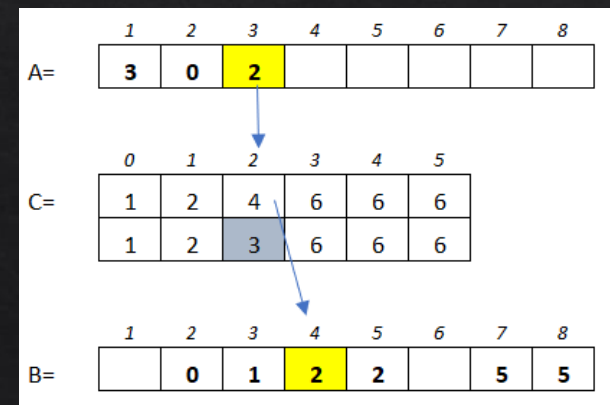
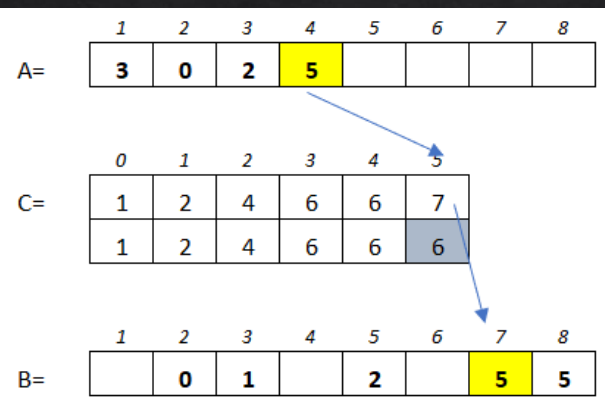
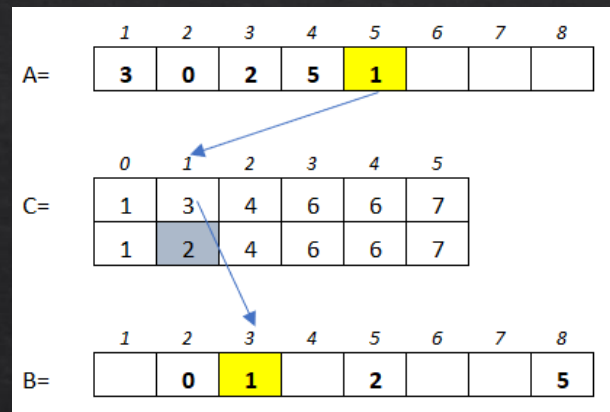
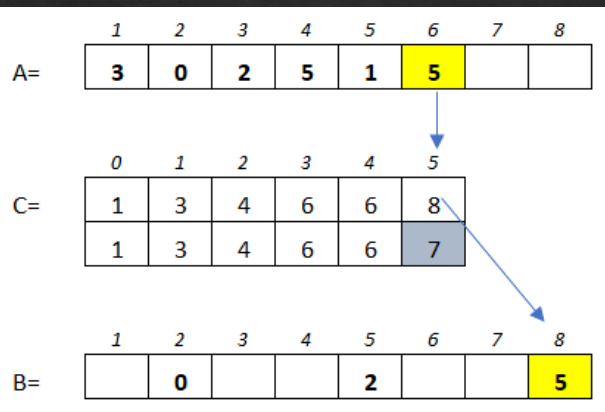
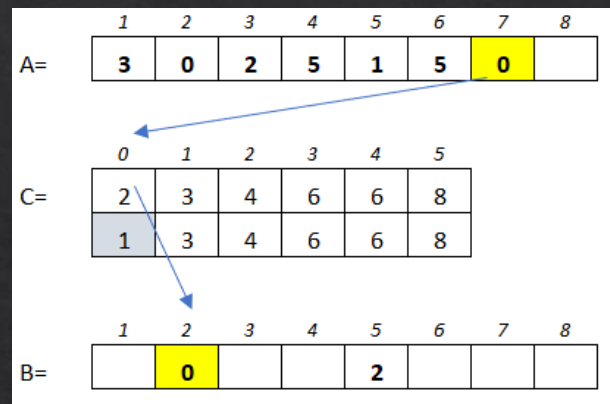
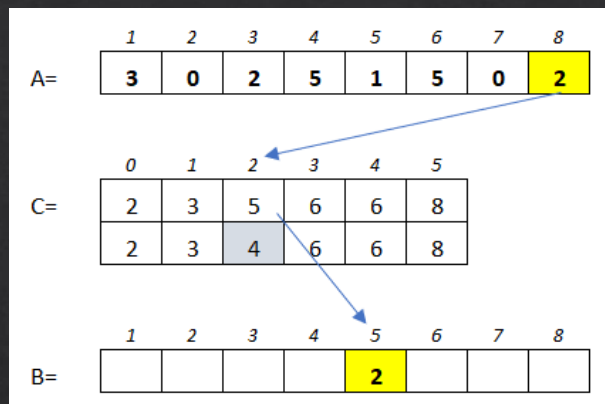
	1	2	3	4	5	6	7	8
A=	3	0	2	5	1	5	0	2

	0	1	2	3	4	5
C=	2	1	2	1	0	2

$C[i]$ indicates how many times i occurs

	0	1	2	3	4	5
C=	2	3	5	6	6	8

Cumulative results



Second step: Move the elements into their proper slot

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$C : \mathbb{N}[r]$ // counter array

$k := 0$ to $r-1$

$C[k] := 0$ // init the counter array

$i := 0$ to $n-1$

$C[\varphi(A[i])]++$ // count the items with the given key

$k := 1$ to $r-1$

$C[k] += C[k-1]$ // $C[k] :=$ the number of items with key $\leq k$

$i := n-1$ downto 0

$k := \varphi(A[i])$ // $k :=$ the key of $A[i]$

$C[k]--$ // The next one with key k must be put before $A[i]$ where
 $B[C[k]] := A[i]$ // Let $A[i]$ be the last of the {unprocessed items with key k }

Iterating over A backwards and putting elements into B results in a stable sort!

Note: On the images the arrays are indexed from 0 but that is not correct, as you can see in the last two instructions in the stuctugram we first subtract, and then use it as an index.

Radix sort on arrays using counting sort

- ◆ Keys as in distribution sort (r, d)
- ◆ We will iterate on the digis from right to left as well, but using counting sort
- ◆ We will use two arrays moving them from one into the other
- ◆ Example: $(r=4, d=2)$:
 $\langle 11, 20, 10, 23, 21, 30 \rangle$

$A = \langle 11, 20, 10, 23, 21, 30 \rangle$

Counting sort on the second digit

		11	20	10	23	21	30			30	21	23	10	20	11
0	0		1	2			3	3	3	2			1	0	
1	0	1				2		2	5		4				3
2	0							0	5						
3	0				1			1	6			5			

Once inserting into it's place we subtract one from the cumulative occurances

B=	1	2	3	4	5	6
	20	10	30	11	21	23

$B = \langle 20, 10, 30, 11, 21, 23 \rangle$

$B = \langle 20, 10, 30, 11, 21, 23 \rangle$

Counting sort on the first digit

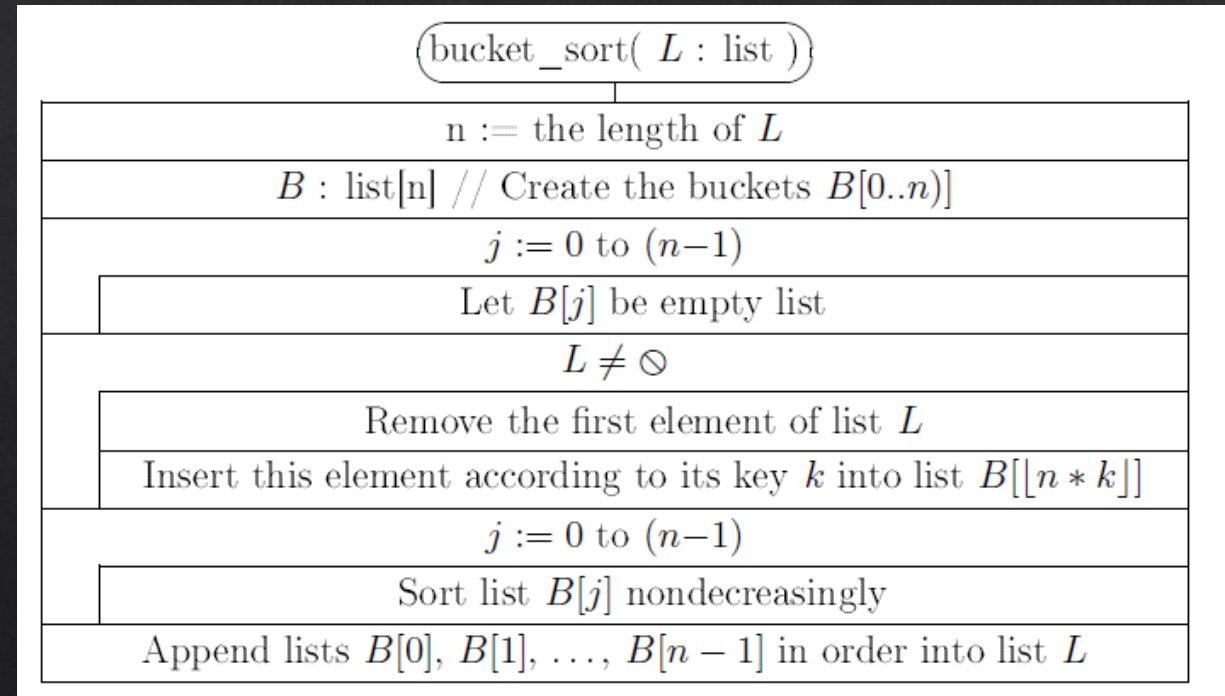
		20	10	30	11	21	23			23	21	11	30	10	20
0	0							0	0						
1	0		1		2			2	2			1		0	
2	0	1				2	3	3	5	4	3				2
3	0			1				1	6				5		

A=	1	2	3	4	5	6
	10	11	20	21	23	30

$A = \langle 10, 11, 20, 21, 23, 30 \rangle$

Bucket sort

- ◆ This time the pre-condition is to have keys from the interval $[0; 1)$.
- ◆ Having n keys to be sorted, create n buckets as well: $B[0..n-1]$
- ◆ Calculate $\lfloor n * k \rfloor$ (bucked index), and insert key into that bucked.
- ◆ Sort the buckets and append one after the other.
- ◆ Of coure now the time complexity is based on how we sort the buckets. The length of the buckets however depend on the distribution of our initial keys. If the keys represent a normal distribution we can have an upper bound in their length and ideally they should be very short lists. See more details in the lecture notes.



Bucket sort demonstration

Input: $\langle 0.79, 0.13, 0.16, 0.64, 0.39, 0.20, 0.89, 0.53, 0.71, 0.42 \rangle$

	After insertion		After sort	For lists containing multiple elements we insert into the front.
0	$\langle \rangle$		$\langle \rangle$	
1	$\langle 0.13 \rangle$	$\langle 0.16, 0.13 \rangle$	$\langle 0.13, 0.16 \rangle$	
2	$\langle 0.20 \rangle$		$\langle 0.20 \rangle$	
3	$\langle 0.39 \rangle$		$\langle 0.39 \rangle$	
4	$\langle 0.42 \rangle$		$\langle 0.42 \rangle$	
5	$\langle 0.53 \rangle$		$\langle 0.53 \rangle$	
6	$\langle 0.64 \rangle$		$\langle 0.64 \rangle$	
7	$\langle 0.79 \rangle$	$\langle 0.71, 0.79 \rangle$	$\langle 0.71, 0.79 \rangle$	
8	$\langle 0.89 \rangle$		$\langle 0.89 \rangle$	
9				

Output: $\langle 0.13, 0.16, 0.20, 0.39, 0.42, 0.53, 0.64, 0.71, 0.79, 0.89 \rangle$

Bucket sort on S1L

bucket_sort(&L : E1*)

n := length(L)
Z : E1*[n] //edények listáinak első elemére mutató pointerok tömbje
j = 0 to n-1
Z[j] := 0;
L ≠ 0
p, L := L, L->next
j := [n * p->key]
p->next := Z[j] Z[j] := p //lista elejére fűzünk
j = 0 to n-1
sort(Z[j])
appendLists(L,Z)

- ◇ Length(L:E1*):N Counts the elements (linear time)
- ◇ Using merge sort: merge(&L:E1*) (n logn time, if upper bound is not accounted)
- ◇ appendLists(&L:E1*, Z:E1*[n]) linear time.

OHW

- ◆ For small number of elements we can make „perfect” comparison based sorting algorithms.
- ◆ For more details check lecture notes.

n	n!	$\lceil \log_2(n!) \rceil$
2	2	1
3	6	3
4	24	5
5	120	7
6	720	10
7	5040	13
8	40320	16
9	362880	19
10	3628800	22

$n=2$



$n=3$



$n=4$

Sorting three elements: max 3 comparisons.
For the 4th element we compare it with the element in the middle, and depending on the result we compare it with its left or right once again = 5 comparisons

$n=5$??

This is the
optional HW